

# FlashPrefill V2: Block-Sparse Prefill Attention for Long-Context LLM Serving

Qihang Fan<sup>1,2,3,\*</sup>, Huaibo Huang<sup>1,2,†</sup>, Zhiying Wu<sup>3</sup>,

Bingning Wang<sup>3,‡</sup>, Ran He<sup>1,2</sup>

<sup>1</sup>MAIS&NLPR, CASIA    <sup>2</sup>UCAS    <sup>3</sup>WeChat, Tencent

 <https://github.com/qhfan/FlashPrefillv2>

## Abstract

Long-context modeling is a pivotal capability for Large Language Models, yet the quadratic complexity of attention remains a critical bottleneck, particularly during the compute-intensive prefilling phase. Our previous work, FlashPrefill [8], mitigates this cost through instantaneous pattern discovery and max-based dynamic thresholding; however, it remains an algorithmic prototype that is still distant from production deployment. In this paper, we present FlashPrefill V2, which evolves FlashPrefill from a prototype toward practical long-context serving along three dimensions. First, we introduce a mean correction term that effectively suppresses the approximation error, keeping performance degradation manageable even at extreme sparsity levels. Second, we redesign the sparse attention operator with PackGQA memory access, warp specialization, and pingpong pipelining, fully aligning with the latest FlashAttention-3/4 implementations [22, 38] and supporting FP8 inference to meet practical quantization requirements. Third, FlashPrefill V2 natively supports paged KV cache and continuous batching, allowing integration as an attention backend in modern inference frameworks such as SGLang [42]. Extensive evaluations on NVIDIA H20 GPUs—among the most widely deployed inference accelerators—demonstrate that FlashPrefill V2 delivers up to **47.26×** and **27.19×** speedups over FlashAttention-2 at 128K context length under FP8 and BF16 precision, respectively, and, in FP8, still achieves a **30.49×** speedup against an FA3/4-aligned dense baseline.

## 1. Introduction

Recent years have witnessed the rapid evolution of Large Language Models (LLMs) [9, 27, 28]. However, the quadratic complexity of self-attention incurs prohibitive overhead on long-context sequences [29], a bottleneck that is particularly salient during the compute-intensive prefill stage. To mitigate this cost, various sparse attention mechanisms have been proposed [11, 14, 35]: they

---

<sup>†</sup> Corresponding Author.

<sup>‡</sup> Project Leader.

\* Work done during internship at WeChat.

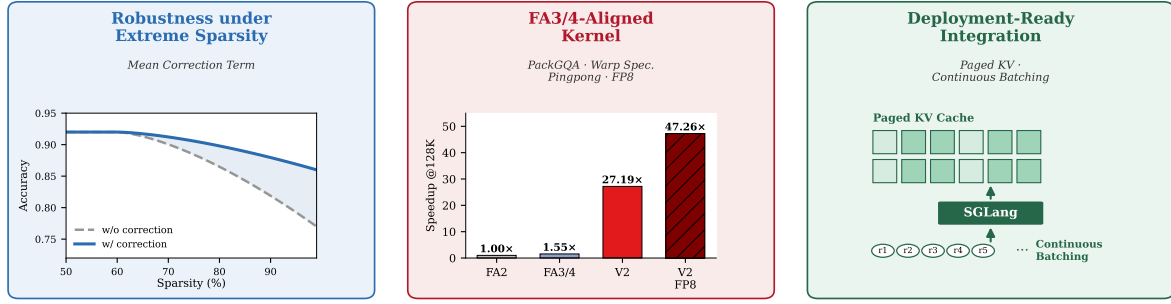


Figure 1 | FlashPrefill V2 evolves FlashPrefill [8] from an algorithmic prototype toward practical long-context serving along three dimensions: (1) a mean correction term that preserves model accuracy under extreme sparsity; (2) an FA3/4-aligned sparse attention kernel with PackGQA memory access, warp specialization, pingpong pipelining, and FP8 support; and (3) native compatibility with paged KV cache and continuous batching for integration as an attention backend in modern serving frameworks such as SGLang [42].

coarsely estimate attention scores to identify salient blocks with Top- $k$ /Top- $p$  selection, and restrict fine-grained computation to these critical tokens.

Our previous work, FlashPrefill [8], pushes this line of research further with instantaneous pattern discovery and max-based dynamic thresholding, eliminating the sorting latency of Top- $k$ /Top- $p$  selection. Nevertheless, it remains an algorithmic prototype that is still distant from production deployment: its accuracy can degrade uncontrollably under aggressive sparsity; its kernel is built upon FlashAttention-2 [5] and lags behind state-of-the-art dense implementations such as FlashAttention-3/4 [22, 38], a gap that is particularly pronounced on Hopper-architecture GPUs where TMA and asynchronous pipelines dominate kernel efficiency; and its contiguous KV layout is incompatible with the paged KV cache and continuous batching that underpin modern serving frameworks [13, 42].

In this paper, we present FlashPrefill V2, which closes these gaps as illustrated in Fig. 1. **(1)** We introduce a mean correction term that compensates pruned blocks with their pooled K/V statistics inside the attention computation, keeping accuracy degradation manageable even at extreme sparsity levels. **(2)** We redesign the sparse attention operator for Hopper-architecture GPUs with PackGQA memory access, warp specialization, and pingpong pipelining, fully aligning with the latest FlashAttention-3/4 implementations and supporting FP8 precision for quantized deployment. **(3)** FlashPrefill V2 natively supports paged KV cache and continuous batching, and can be integrated as an attention backend in modern inference frameworks such as SGLang.

As shown in Fig. 2, on NVIDIA H20, one of the most widely deployed inference GPUs and a representative of the Hopper architecture, FlashPrefill V2 attains a **27.19 $\times$**  speedup over FlashAttention-2 at a 128K context length under BF16 precision, surpassing all competing sparse operators, while its FP8 variant further reaches **47.26 $\times$** . Even against the much stronger FA3/4-aligned dense baseline, FlashPrefill V2 still delivers speedups of **17.54 $\times$**  (BF16) and **30.49 $\times$**  (FP8), and maintains acceleration at sequence lengths as short as 4K.

Our contributions can be summarized as follows:

- A mean correction term for block-wise score estimation that ensures controlled accuracy degradation even under extreme sparsity.
- A redesigned sparse attention operator featuring PackGQA memory access, warp special-

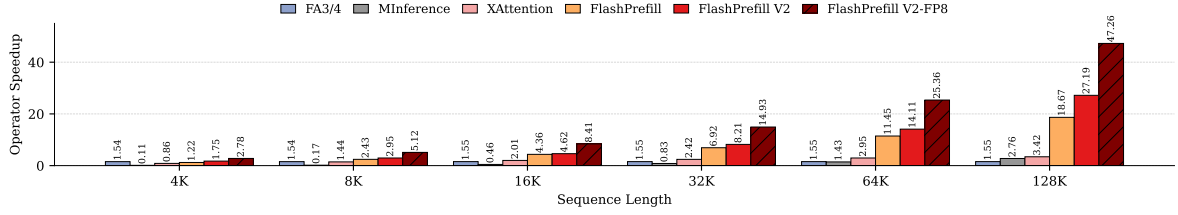


Figure 2 | Speedup of various attention operators relative to FlashAttention-2 [5] on NVIDIA H20 GPUs, including an FA3/4-aligned dense baseline [22, 38]. All results are measured with a batch size of 4. FlashPrefill V2 exhibits a dominant advantage, particularly in long-context scenarios, and its FP8 variant further amplifies the gains.

ization, and pingpong pipelining, fully aligned with FlashAttention-3/4 and extended to FP8 inference, attaining up to 47.26× speedup over FlashAttention-2 at 128K context.

- Native compatibility with paged KV cache and continuous batching, demonstrated through integration into SGLang with up to 4.8× lower end-to-end time-to-first-token at 128K with marginal accuracy loss.

## 2. Related Works

**Sparse Attention.** Reducing the quadratic cost of attention via sparsity has been extensively studied. Early designs fix the sparse layout in advance, via local windows with global tokens [3, 39] or hashing- and clustering-based routing [12, 30], or evict KV entries under a preset budget [4, 16, 31, 32, 40]. Learnable variants integrate sparsity into training or fine-tuning, coupling the pattern with the weights [18, 34, 37, 41]. Training-free methods instead estimate which regions of the attention map are worth computing and skip the rest at inference [11, 14, 35], extend to query-aware KV selection during decoding [21, 23, 25, 33, 44], and to multimodal inputs [17]. A recent variant compensates pruned blocks with surrogate contributions [15], and UniPrefill [7] lifts block-wise sparsification to token-level computation for hybrid architectures. FlashPrefill V2 belongs to the training-free category: compared with prior training-free methods, it further emphasizes controlled accuracy under extreme sparsity and system-level deployability, rather than treating the sparse pattern as the sole design axis. A complementary line replaces softmax attention altogether with linear or recurrent formulations [19, 24], e.g., rectifying its neglect of query magnitude [6].

**Efficient Attention Kernels.** Orthogonal to sparsity, a parallel line of work accelerates dense attention through careful hardware-aware kernel design. FlashAttention-2 reformulates the computation around better parallelism and work partitioning [5], while FlashAttention-3 exploits asynchrony and low-precision arithmetic on Hopper GPUs [22], and FlashAttention-4 co-designs the algorithm with kernel pipelining to accommodate asymmetric hardware scaling [38]. Our sparse operator inherits this design philosophy: built upon CUTLASS/CuTe with TMA and warp-specialized producer-consumer pipelines, it adopts PackGQA memory access so that block sparsity translates into wall-clock speedup rather than being offset by kernel inefficiency, and it further supports FP8 execution.

**LLM Serving Systems.** Production-grade inference increasingly relies on system-level innovations in memory management and scheduling. vLLM popularized paged KV cache management with PagedAttention [13], SGLang optimizes the execution of structured language model programs [42], and disaggregated architectures further separate prefill from decoding to satisfy

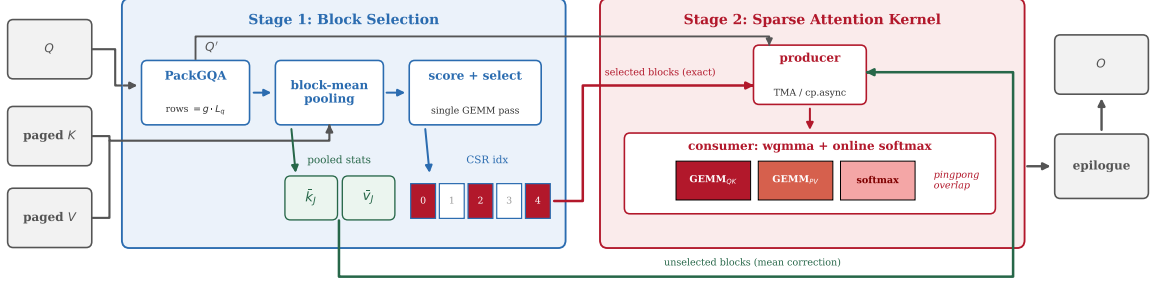


Figure 3 | The FlashPrefill V2 prefill pipeline. Stage 1 packs queries with PackGQA, pools block-mean statistics, and produces a CSR index of selected blocks in a single fused pass; Stage 2 runs the warp-specialized sparse attention kernel over the selected blocks (exact path) while the unselected blocks are compensated through the mean correction path (Sec. 3.2), and the two streams merge inside the online softmax.

latency objectives [20, 43]; continuous batching across concurrent requests has since become a standard technique for maximizing throughput. However, most research-oriented sparse attention kernels assume a contiguous KV layout, which prevents direct adoption in these frameworks. FlashPrefill V2 is instead designed around paged KV cache and continuous batching from the outset, and can be integrated as an attention backend in modern serving engines.

### 3. Method

#### 3.1. Preliminaries: The FlashPrefill Framework

We first recap the core computation of FlashPrefill [8], upon which FlashPrefill V2 is built. Consider the prefilling stage with  $L$  tokens, where  $Q, K, V \in \mathbb{R}^{L \times d}$  denote the query, key, and value matrices of one attention head, and the causal output is  $O = \text{softmax}(QK^\top / \sqrt{d} + \mathcal{M})V$ . FlashPrefill partitions  $K$  and  $V$  into blocks of size  $B$  and accelerates prefilling in two stages.

**Block-level Score Estimation.** FlashPrefill identifies the salient blocks with uniformly distributed probe queries, which simultaneously resolve vertical, slash, and block-sparse patterns. Each key block  $\mathcal{B}_J$  is represented by its mean-pooled key  $\bar{k}_J = \frac{1}{B} \sum_{k_i \in \mathcal{B}_J} k_i$ . Since  $\exp(q \cdot \bar{k}_J)$  is the geometric mean of the per-token scores  $\{\exp(q \cdot k_i)\}$ , by the AM-GM inequality it lower-bounds the block’s true contribution  $\frac{1}{B} \sum_i \exp(q \cdot k_i)$ , while the low intra-block variance of attention distributions keeps the cross-block ranking preserved. To avoid materializing the  $L \times (L/B)$  score matrix, a fused kernel computes, for each query tile  $I$  and key block  $J$ ,

$$m_{I,J} = \max_{q_i \in I} (q_i \cdot \bar{k}_J), \quad \mathcal{S}_{I,J} = \sum_{q_i \in I} \exp(q_i \cdot \bar{k}_J - m_{I,J}), \quad (1)$$

followed by a global rescaling with  $\mathcal{M}_I = \max_J m_{I,J}$  and row normalization:

$$\text{Score}_{I,J} = \frac{\mathcal{S}_{I,J} \cdot \exp(m_{I,J} - \mathcal{M}_I)}{\sum_{J'} \mathcal{S}_{I,J'} \cdot \exp(m_{I,J'} - \mathcal{M}_I) + \epsilon'}, \quad (2)$$

reducing the memory footprint from  $O(L^2/B)$  to  $O((L/B)^2)$ .

**Max-based Dynamic Thresholding.** Instead of Top- $k$ /Top- $p$  selection, which requires global sorting or cumulative summation, FlashPrefill derives the pruning threshold for each query

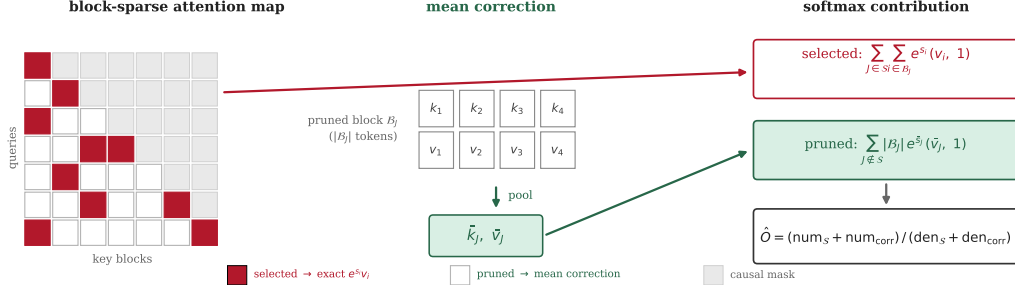


Figure 4 | Mean correction. Selected blocks (red) are computed exactly, while each pruned block is pooled into its mean statistics  $(\bar{k}_J, \bar{v}_J)$  and contributes a surrogate term  $|\mathcal{B}_J|e^{\bar{s}_J}(\bar{v}_J, 1)$  to the softmax numerator and denominator, recovering the discarded probability mass without per-token computation.

block  $I$  from its peak score,

$$\text{thresh}_I = \alpha \cdot \max_J \text{Score}_{I,J}, \quad (3)$$

and retains every key block with  $\text{Score}_{I,J} \geq \text{thresh}_I$ , together with the first sink blocks, a local sliding-window band, and the most recent blocks, subject to the causal constraint. Block-sparse attention is then executed only over the selected blocks.

### 3.2. Mean Correction under Extreme Sparsity

For a query  $q$  with per-token logits  $s_i = q \cdot k_i / \sqrt{d}$ , let  $\mathcal{S}$  denote the selected block set. FlashPrefill approximates the exact output by truncating both sums of the softmax to  $\mathcal{S}$ :

$$O = \frac{\sum_J \sum_{i \in \mathcal{B}_J} e^{s_i} v_i}{\sum_J \sum_{i \in \mathcal{B}_J} e^{s_i}} \approx \frac{\sum_{J \in \mathcal{S}} \sum_{i \in \mathcal{B}_J} e^{s_i} v_i}{\sum_{J \in \mathcal{S}} \sum_{i \in \mathcal{B}_J} e^{s_i}}. \quad (4)$$

Under extreme sparsity, however, the discarded mass  $\sum_{J \notin \mathcal{S}} \sum_{i \in \mathcal{B}_J} e^{s_i}$  can no longer be neglected. We recover it with a zero-order mean correction (a surrogate-contribution idea also adopted in [15]), as illustrated in Fig. 4: for each key block  $J$ , we precompute

$$\bar{k}_J = \frac{1}{|\mathcal{B}_J|} \sum_{i \in \mathcal{B}_J} k_i, \quad \bar{v}_J = \frac{1}{|\mathcal{B}_J|} \sum_{i \in \mathcal{B}_J} v_i, \quad \bar{s}_J = q \cdot \bar{k}_J / \sqrt{d}, \quad (5)$$

and the corrected output reads

$$\hat{O} = \frac{\sum_{J \in \mathcal{S}} \sum_{i \in \mathcal{B}_J} e^{s_i} v_i + \sum_{J \notin \mathcal{S}} |\mathcal{B}_J| e^{\bar{s}_J} \bar{v}_J}{\sum_{J \in \mathcal{S}} \sum_{i \in \mathcal{B}_J} e^{s_i} + \sum_{J \notin \mathcal{S}} |\mathcal{B}_J| e^{\bar{s}_J}}. \quad (6)$$

**Approximation error.** Writing  $\delta_i = s_i - \bar{s}_J$  with  $\sum_i \delta_i = 0$  and denoting within-block averages by  $\overline{(\cdot)}$ , the exact mass and weighted value of block  $J$  expand around  $\bar{s}_J$  as

$$\sum_{i \in \mathcal{B}_J} e^{s_i} = |\mathcal{B}_J| e^{\bar{s}_J} \left( 1 + \frac{\overline{\delta^2}}{2} + O(\overline{\delta^3}) \right), \quad \sum_{i \in \mathcal{B}_J} e^{s_i} v_i = |\mathcal{B}_J| e^{\bar{s}_J} \left( \bar{v}_J + \overline{\delta v} + O(\overline{\delta^2 v}) \right), \quad (7)$$

where  $\overline{\delta v} = \frac{1}{|\mathcal{B}_J|} \sum_i \delta_i (v_i - \bar{v}_J)$  is the within-block covariance between logit deviations and values: the mass surrogate is second-order accurate, whereas the numerator surrogate retains a first-order covariance term. With block mass  $w_J = \sum_{i \in \mathcal{B}_J} e^{s_i}$ , total mass  $D = \sum_J w_J$ , and exact block mean  $\mu_J = w_J^{-1} \sum_{i \in \mathcal{B}_J} e^{s_i} v_i$ , substituting the surrogate for a pruned block shifts the output by

$$\Delta_J = \frac{w_J}{D} (\mu_J - \bar{v}_J) + O\left(\frac{w_J}{D} \overline{\delta^2}\right), \quad \mu_J - \bar{v}_J = \overline{\delta v} + O(\overline{\delta^2 v}). \quad (8)$$

Eq. 8 is controlled by three bounds. (i) Max-based thresholding (Sec. 3.1) caps the mass share of every pruned block,  $w_J/D \leq \alpha$ , so the total shift is a small-share signed sum,

$$\left\| \sum_{J \notin S} \Delta_J \right\| \leq \rho_{\text{pr}} \cdot \max_{J \notin S} \|\mu_J - \bar{v}_J\|, \quad \rho_{\text{pr}} = \sum_{J \notin S} \frac{w_J}{D}. \quad (9)$$

(ii) The per-block deviation is a covariance, vanishing under within-block independence and bounded by dispersion via Cauchy–Schwarz,

$$\overline{\delta v} = 0 \text{ if } \delta \perp (v - \bar{v}_J), \quad \|\overline{\delta v}\| \leq \delta_{\text{rms}} \cdot \sigma_v, \quad (10)$$

where  $\sigma_v$  is the within-block value dispersion, itself small under the low intra-block variation that justifies block-mean score estimation (Sec. 3.1). (iii) Relative to dropping the block outright,  $\Delta_J^{\text{none}} = \frac{w_J}{D} (\mu_J - O)$ , the correction cancels the zeroth order and retains only the covariance:

$$\frac{\|\Delta_J\|}{\|\Delta_J^{\text{none}}\|} = \frac{\|\mu_J - \bar{v}_J\|}{\|\mu_J - O\|} \sim O(\delta_{\text{rms}}), \quad (11)$$

since  $\mu_J - O$  is an inter-block deviation while  $\mu_J - \bar{v}_J$  is an intra-block one. Salient blocks always follow the exact path, so only the small- $w_J$  tail enters Eq. 9. Empirically (Tab. 8), removing the correction costs up to 6.2 points at 128K in FP8, while the corrected pipeline stays within 1.5 points of full attention even at 128K in BF16.

**Implementation.** As illustrated by the green correction path in Fig. 3, only blocks fully visible to the whole query tile are corrected; blocks intersecting the diagonal band are covered by the exact sink/window/recent selections. Within the kernel, each corrected block enters the pipeline as one extra iteration whose score is shifted by  $\log |\mathcal{B}_J|$ , so that a single mean vector represents  $|\mathcal{B}_J|$  tokens in both the numerator and the denominator of Eq. 6, reusing the same MMA pipeline and online-softmax rescaling without any additional kernel launch. Concretely, a two-pointer scan over the selected-block list aggregates the correction targets of each chunk into a bitmask, `todo = valid &  $\neg$  selected`, and chunks in which every block is selected are skipped identically by the producer and the consumer, so the pipeline stays in lockstep without extra synchronization.

### 3.3. Hopper-Aligned Sparse Attention Operator

The V1 sparse kernel inherits the execution model of FlashAttention-2, whose tile-based synchronous pipeline under-utilizes Hopper GPUs: the Tensor Memory Accelerator (TMA) and asynchronous warpgroup MMAs (`wgmma`) only reach full throughput when data movement and computation are decoupled. We therefore redesign the operator on top of a CUTLASS/CuTe warp-specialized pipeline (Stage 2 of Fig. 3). Our implementation builds upon the Hopper-specific techniques established by FlashAttention-3/4 themselves [22, 38]—TMA-based asynchronous data movement, warp-specialized producer-consumer pipelines, and intra-warpgroup GEMM-softmax overlap—and our contribution lies in extending this execution model to block-sparse computation (index-driven traversal, correction chunk injection) and to the GQA/paged/FP8 regimes that deployment requires, as detailed below.

**PackGQA Memory Layout.** Under grouped-query attention with ratio  $g = H_q/H_{kv}$ , a naive mapping assigns one thread block to each (query head,  $M$ -tile) pair, so that  $g$  thread blocks repeatedly load the same KV block. We instead reshape the query matrix as

$$Q \in \mathbb{R}^{L \times H_q \times d} \longrightarrow Q' \in \mathbb{R}^{(g \cdot L) \times H_{kv} \times d}, \quad (12)$$

where the packed row index  $r$  maps to position and head via

$$\text{pos}(r) = \lfloor r/g \rfloor, \quad h(r) = h_{kv} \cdot g + (r \bmod g), \quad (13)$$

implemented with a single fast divmod per row. Each  $M$ -tile of  $Q'$  covers  $k\text{Block}M/g$  positions of all  $g$  heads of one KV group, yielding three properties: (i) each staged KV block is consumed by all rows of the tile, so group-wide sharing is guaranteed in shared memory and never relies on cross-CTA L2 retention; (ii) the sparse index of Sec. 3.1 is defined per KV head, shrinking the index metadata by a factor of  $g$ ; and (iii) tiles stay fully occupied for any  $L$ : the wasted-row fraction is  $(\lceil L/k\text{Block}M \rceil k\text{Block}M - L)/(\lceil L/k\text{Block}M \rceil k\text{Block}M)$  per head under the naive mapping but only  $(\lceil gL/k\text{Block}M \rceil k\text{Block}M - gL)/(\lceil gL/k\text{Block}M \rceil k\text{Block}M)$  once per group under packing, which for  $L \leq k\text{Block}M/g$  (e.g., decode) cuts KV stagings by up to the full factor of  $g$ .  $Q$  is staged with vectorized `cp.async` loads, with threads of the same row sharing one warp to amortize page-table lookups.

**Warp-Specialized Producer-Consumer Pipeline.** The persistent kernel partitions its warp-groups into one producer warpgroup issuing asynchronous TMA/`cp.async` loads of K/V (and of the correction chunks of Sec. 3.2) into a multi-stage shared-memory pipeline, and two consumer warpgroups executing wgmma on staged tiles, with registers reallocated between the two roles (`warpgroup_reg_dealloc/alloc`). On top of this, GEMM and softmax proceed in a pingpong fashion within each consumer warpgroup: each step issues the QK GEMM of block  $n$ , the PV GEMM of block  $n+1$ , and the online softmax of block  $n$ ,

$$\underbrace{S^{(n)} = Q K_{(n)}^\top}_{\text{GEMM}_0} \parallel \underbrace{O += P^{(n+1)} V_{(n+1)}}_{\text{GEMM}_1} \parallel \underbrace{\tilde{P}^{(n)} = \text{online-softmax}(S^{(n)})}_{\text{softmax}}, \quad (14)$$

so that MMA and non-MMA (exponentiation, rescaling) units overlap instead of stalling each other. This matches the intra-warpgroup overlap schedule of state-of-the-art dense implementations [22, 38], which our dense path replicates to provide a strong baseline.

**FP8 Execution.** With per-tensor scales  $c_q, c_k, c_v$ , FP8-e4m3 operands are dequantized on the fly inside the online softmax:

$$S = c_q c_k \cdot (\tilde{Q} \tilde{K}^\top), \quad \tilde{P} = \lfloor 2^8 \cdot \exp(S - m) \rfloor_{\text{e4m3}} \in [0, 256], \quad O = c_v \cdot \frac{\sum_t \tilde{P}^{(t)} \tilde{V}^{(t)}}{\sum_t \tilde{P}^{(t)}}, \quad (15)$$

where  $\lfloor \cdot \rfloor_{\text{e4m3}}$  denotes FP8 quantization and  $m$  the running row maximum. The  $2^8$  offset maps probabilities onto  $[0, 256]$ , fully utilizing the e4m3 dynamic range; it cancels in the softmax ratio and is recovered only in the log-sum-exp at finalize.

The second GEMM requires special handling because FP8 wgmma mandates a K-major (transposed) layout for the B operand, which constrains how  $\tilde{V}$  is fed:

- **Column-major**  $\tilde{V}$  already satisfies the layout and is consumed with zero copying; the probability fragment  $\tilde{P}$  is instead re-arranged into the wgmma A-operand register layout purely in registers via warp shuffles and byte permutation, and the output columns are un-permuted symmetrically in the epilogue:

$$\tilde{P} \xrightarrow{\text{shfl+byte\_perm}} \tilde{P}', \quad O' = \tilde{P}' \tilde{V} \xrightarrow{\text{epilogue}} O. \quad (16)$$

- **Row-major**  $\tilde{V}$  is transposed on the fly by the producer inside shared memory with an LDSM.T→byte-permute→STSM sequence; to avoid bank conflicts we permute the columns of  $\tilde{V}$  during this transpose and restore the column order of  $O$  in the epilogue, rather than transposing conflict-free at full price.

In both cases the layout conversions involve no global-memory traffic, and the PV GEMM consumes  $\tilde{P}$  directly from registers (RS-mode wgmma).

**Index-Driven Sparse Traversal.** The selection result is stored as a CSR index: for each (batch, head, tile) segment  $g$  with offsets  $\text{cu}[g]$  and  $T = \text{cu}[g+1] - \text{cu}[g]$ , the ascending index list  $\{\text{idx}[\text{cu}[g] + i]\}_{i=0}^{T-1}$  defines the iterated block sequence

$$n^{(t)} = \text{idx}[\text{cu}[g] + T - 1 - t], \quad t = 0, \dots, T - 1, \quad (17)$$

traversed backwards to match the descending order of the dense schedule. Producer and consumer evaluate the same sequence  $\{n^{(t)}\}$  independently, so their control flows are identical by construction and the pipeline advances in lockstep without any synchronization for block skipping; the next index  $n^{(t+1)}$  is prefetched while the load and GEMM of  $n^{(t)}$  are still in flight, keeping index resolution off the critical path. Under KV splitting, split  $u$  of  $U$  receives the contiguous subsequence

$$t \in [u \cdot \lceil T/U \rceil, \min((u+1)\lceil T/U \rceil, T)), \quad (18)$$

i.e., the CSR list is partitioned by selected-block count rather than by position range, equalizing actual work across splits.

**Single-Pass Score and Selection.** Scoring and thresholding are fused into one pass over the pooled keys. With  $s_{iJ} = q_i \cdot \tilde{k}_J / \sqrt{d}$ , the scoring kernel maintains a running tile maximum  $M$  and block energies online,

$$M \leftarrow \max(M, M_{\text{tile}}), \quad E_J = \sum_{q_i \in I} 2^{s_{iJ} \log_2 e - M}, \quad (19)$$

then writes  $E_J$  (bitcast to integer) into the index buffer itself together with the per-chunk maximum, eliminating both a separate score buffer and a second GEMM pass. Selection then only rescales and thresholds:

$$\hat{E}_J = E_J \cdot 2^{M_c - M_{\text{final}}}, \quad J \in \mathcal{S} \iff \hat{E}_J \geq \alpha \cdot \max_{J'} \hat{E}_{J'}, \quad \forall J \in \{\text{sink}\} \cup \{\text{window}\} \cup \{\text{diag}\}, \quad (20)$$

and survivors are compacted in place, which is safe since the write position never exceeds the scan position.

**Sparsity-Aware Load Balancing.** After PackGQA, the cost of tile  $I$  is proportional to its selected-block count,

$$C(I) \propto T_I = O(L) \text{ for the last tiles,} \quad O(1) \text{ otherwise,} \quad (21)$$

so the dense heuristic that disables splitting once tiles cover the SMs leaves the heavy tail tiles serialized. We therefore always admit splits on the block-sparse path: requests are scheduled in descending order of their causal workload by a device-side sort in the scheduler preparation pass, and each tile's selected-block list is partitioned evenly across splits by count (Eq. 18), so that every split receives an equal share of the actual sparse work; tile assignment is resolved by warp-level parallel prefix scans over per-request lengths.

---

**Algorithm 1** FlashPrefill V2 prefill pipeline (one attention layer)

---

**Require:** packed queries  $Q$ , paged KV cache  $(K, V)$  with page table, lengths  $\{(L_b^q, L_b^{kv})\}$ , threshold  $\alpha$

**Ensure:** attention output  $O$

- 1:  $Q' \leftarrow \text{PackGQA}(Q)$ , with row mapping of Eq. 13
  - 2:  $(\bar{k}_J, \bar{v}_J) \leftarrow$  block means of  $(K, V)$ , one Triton pass (fp8 descaled if needed)
  - 3:  $E_J, M \leftarrow$  fused scoring of Eq. 19 over  $(Q', \bar{k})$  // single GEMM pass
  - 4:  $S \leftarrow$  thresholding and in-place compaction of Eq. 20, forcing sink/window/diag blocks
  - 5:  $(\text{cu}, \text{idx}) \leftarrow$  CSR index of  $S$ , logical blocks expanded to  $B/64$  physical tiles
  - 6: **for** each work unit  $(b, h, I)$  from the persistent scheduler (descending workload order) **do**
  - 7:   producer: stage  $K, V$  tiles of  $\{n^{(t)}\}$  (Eq. 17) via TMA/cp.async through the page table (Eq. 22)
  - 8:   consumer: wgmma + online softmax in pingpong order (Eq. 14)
  - 9:   correction: append mean chunks of unselected blocks with logit shift  $\log |\mathcal{B}_J|$  (Eq. 6)
  - 10: **end for**
  - 11: epilogue: rescale by the accumulated softmax denominator (and un-permute for fp8);
  - 12: **return**  $O$
- 

**Paged and Variable-Length Execution.** K/V addressing goes through the page table: the global address of key token  $j$  of request  $b$  is

$$\text{addr}(b, j) = \text{page\_table}[b, \lfloor j/p \rfloor] \cdot p + (j \bmod p), \quad (22)$$

with page size  $p$ , resolved on the fly with `cp.async`; a varlen-aware persistent scheduler enumerates  $(b, h, I)$  work units from per-request lengths  $(L_b^q, L_b^{kv})$ , which is exactly the execution model of continuous batching. The CSR index format is compatible with SGLang, allowing direct integration as an attention backend (Sec. 4).

**Implementation Details.** All instantiations share one tile configuration ( $128 \times 64$ , two-stage smem pipeline,  $B/64$  physical tiles per logical block); registers are statically reallocated between roles: the producer holds 24 registers with TMA-based KV (40 on the `cp.async` paged path), and each consumer warpgroup holds 240 (232 on the paged path).

### 3.4. End-to-End Prefill Pipeline

Fig. 3 and Algorithm 1 put the pieces together. Given a batch of requests with new-token lengths  $\{L_b^q\}$  and cached KV lengths  $\{L_b^{kv}\}$ , FlashPrefill V2 executes two stages per layer: an index stage (Sec. 3.1) that packs queries, scores pooled blocks, and compacts the survivors into a CSR index, and an attention stage (Sec. 3.2, 3.3) that runs the warp-specialized sparse operator over the selected blocks with mean correction.

**Overhead Analysis.** For one KV head with group size  $g$ , query length  $L_q$ , and KV length  $L$ , the FLOP costs of the three components are

$$C_{\text{idx}} = O\left(gL_q \cdot \frac{L}{B} \cdot d\right), \quad C_{\text{corr}} = O\left(gL_q \cdot \frac{(1-\rho)L}{B} \cdot d\right), \quad C_{\text{attn}} = O(gL_q \cdot \rho L \cdot 2d), \quad (23)$$

where  $\rho$  is the retained density,  $B$  the block size, and the factor of two accounts for the QK and PV GEMMs. Both the index stage and the correction operate on one pooled vector per block, hence a factor of  $B$  below the dense reference; relative to the sparse attention itself,

$$\frac{C_{\text{idx}} + C_{\text{corr}}}{C_{\text{attn}}} = O\left(\frac{2-\rho}{2\rho B}\right), \quad (24)$$

| Method                             | RULER Score |       |       |       |       |       |              | Operator Speedup over FA2 |              |              |               |               |               |
|------------------------------------|-------------|-------|-------|-------|-------|-------|--------------|---------------------------|--------------|--------------|---------------|---------------|---------------|
|                                    | 4K          | 8K    | 16K   | 32K   | 64K   | 128K  | Avg.         | 4K                        | 8K           | 16K          | 32K           | 64K           | 128K          |
| <i>Llama-3.1-8B-Instruct</i>       |             |       |       |       |       |       |              |                           |              |              |               |               |               |
| Full                               | 95.98       | 94.74 | 93.07 | 89.06 | 86.27 | 73.82 | 88.82        | 1.00×                     | 1.00×        | 1.00×        | 1.00×         | 1.00×         | 1.00×         |
| MIInference                        | 95.06       | 94.23 | 92.18 | 87.68 | 83.01 | 71.61 | 87.30        | 0.12×                     | 0.18×        | 0.46×        | 0.83×         | 1.34×         | 2.45×         |
| FlexPrefill                        | 95.18       | 94.09 | 92.35 | 86.79 | 84.08 | 72.01 | 87.42        | 0.12×                     | 0.33×        | 0.98×        | 2.21×         | 4.16×         | 5.18×         |
| XAttention                         | 95.24       | 94.37 | 91.61 | 87.03 | 83.65 | 72.33 | 87.37        | 0.79×                     | 1.29×        | 1.83×        | 2.34×         | 3.19×         | 3.48×         |
| FlashPrefill                       | 95.16       | 94.29 | 92.11 | 87.49 | 84.18 | 70.91 | 87.36        | 1.21×                     | 2.38×        | 4.31×        | 7.46×         | 13.62×        | 22.67×        |
| FlashPrefill V2                    | 95.52       | 94.65 | 92.95 | 88.48 | 83.07 | 72.08 | <b>87.79</b> | <b>1.66×</b>              | <b>2.62×</b> | <b>4.28×</b> | <b>7.63×</b>  | <b>14.39×</b> | <b>27.21×</b> |
| FlashPrefill V2-FP8*               | 95.28       | 94.48 | 92.78 | 88.80 | 80.28 | 67.78 | 86.57        | 2.67×                     | 4.56×        | 7.79×        | 13.89×        | 25.76×        | 47.33×        |
| <i>Qwen3-4B-Instruct-2507</i>      |             |       |       |       |       |       |              |                           |              |              |               |               |               |
| Full                               | 94.82       | 92.98 | 91.68 | 88.86 | 82.81 | 71.22 | 87.06        | 1.00×                     | 1.00×        | 1.00×        | 1.00×         | 1.00×         | 1.00×         |
| MIInference                        | 94.06       | 92.18 | 91.09 | 88.07 | 82.08 | 68.01 | 85.92        | 0.11×                     | 0.15×        | 0.42×        | 0.83×         | 1.32×         | 2.53×         |
| FlexPrefill                        | 93.81       | 90.96 | 91.41 | 87.62 | 81.37 | 67.24 | 85.40        | 0.12×                     | 0.33×        | 0.99×        | 2.28×         | 4.01×         | 5.27×         |
| XAttention                         | 94.42       | 91.49 | 91.42 | 87.31 | 81.56 | 68.24 | 85.74        | 0.81×                     | 1.35×        | 1.92×        | 2.38×         | 3.07×         | 3.43×         |
| FlashPrefill                       | 94.71       | 92.04 | 91.28 | 87.29 | 81.08 | 69.02 | 85.90        | 1.21×                     | 2.40×        | 4.36×        | 6.91×         | 11.54×        | 19.26×        |
| FlashPrefill V2                    | 94.49       | 92.42 | 91.56 | 87.48 | 81.67 | 69.76 | <b>86.23</b> | <b>1.72×</b>              | <b>2.82×</b> | <b>4.63×</b> | <b>7.92×</b>  | <b>13.93×</b> | <b>27.45×</b> |
| FlashPrefill V2-FP8                | 94.13       | 92.37 | 91.36 | 87.47 | 80.59 | 68.97 | <b>85.82</b> | <b>2.75×</b>              | <b>4.87×</b> | <b>8.40×</b> | <b>14.44×</b> | <b>25.06×</b> | <b>47.59×</b> |
| <i>Qwen3-30B-A3B-Instruct-2507</i> |             |       |       |       |       |       |              |                           |              |              |               |               |               |
| Full                               | 95.78       | 95.23 | 94.36 | 92.72 | 90.11 | 84.12 | 92.05        | 1.00×                     | 1.00×        | 1.00×        | 1.00×         | 1.00×         | 1.00×         |
| MIInference                        | 95.71       | 94.33 | 93.81 | 92.01 | 89.02 | 82.09 | 91.16        | 0.11×                     | 0.17×        | 0.46×        | 0.83×         | 1.43×         | 2.76×         |
| FlexPrefill                        | 95.32       | 94.71 | 93.52 | 91.81 | 88.09 | 83.11 | 91.09        | 0.12×                     | 0.34×        | 0.98×        | 2.31×         | 3.92×         | 5.43×         |
| XAttention                         | 95.43       | 94.12 | 92.78 | 91.06 | 88.39 | 82.23 | 90.67        | 0.86×                     | 1.44×        | 2.01×        | 2.42×         | 2.95×         | 3.42×         |
| FlashPrefill                       | 95.62       | 94.47 | 93.28 | 92.17 | 88.14 | 82.96 | 91.11        | 1.22×                     | 2.43×        | 4.36×        | 6.92×         | 11.45×        | 18.67×        |
| FlashPrefill V2                    | 95.35       | 94.85 | 94.19 | 92.48 | 89.88 | 83.83 | <b>91.76</b> | <b>1.75×</b>              | <b>2.95×</b> | <b>4.62×</b> | <b>8.21×</b>  | <b>14.11×</b> | <b>27.19×</b> |
| FlashPrefill V2-FP8                | 95.28       | 94.53 | 94.12 | 92.34 | 89.54 | 82.55 | <b>91.39</b> | <b>2.78×</b>              | <b>5.12×</b> | <b>8.41×</b> | <b>14.93×</b> | <b>25.36×</b> | <b>47.26×</b> |

Table 1 | RULER scores (left) and attention operator speedups over FlashAttention-2 (right) at sequence lengths from 4K to 128K. The speedups are measured end-to-end on needle-in-a-haystack inputs, including pattern discovery, thresholding, and mean correction. Results marked with \* (in gray) are measured with direct online FP8 quantization without corrected weights.

i.e., the auxiliary FLOPs vanish as the retained density grows, and even at  $\rho = 4\%$  with  $B = 128$  they are bounded by roughly 19%, consistent with the gap between the attention-only and end-to-end speedups at long contexts in Fig. 2.

**Serving Integration.** FlashPrefill V2 is integrated into SGLang as a standard attention backend. The two-stage pipeline runs in the extend (prefill) phase, while decode falls back to dense attention, since a single query token per step leaves no room for block-level sparsity. The page-level index required by our kernels is derived from SGLang’s token-level mapping by striding every  $p$  entries, and per-forward metadata is built once per step and shared by all layers. Selection hyperparameters ( $\alpha$ , sink/window sizes) are exposed as server arguments, and the index stage reuses a per-stream workspace cache whose capacity-matched reuse avoids any host synchronization or allocator round trip in steady state. No changes to the model definition, the KV cache layout, or the scheduling logic are required.

## 4. Experiments

We evaluate FlashPrefill V2 on two widely-recognized long-context benchmarks, RULER [10] and LongBench [2], across representative LLMs. We first describe the experimental setup, then report accuracy and efficiency results, and finally present ablation studies on the individual components.

| Method                      | Single-Document QA |        |       | Multi-Document QA |        |       |         | Summarization |        |       |       | Few-shot Learning |       |       |        | Synthetic |       | Code   |        | Avg.  |       |       |
|-----------------------------|--------------------|--------|-------|-------------------|--------|-------|---------|---------------|--------|-------|-------|-------------------|-------|-------|--------|-----------|-------|--------|--------|-------|-------|-------|
|                             | NanoQA             | Qasper | MP-en | MP-zh             | Hotpot | 2Wiki | MusiQue | DuRead        | GovRep | OMSum | MNews | VCsum             | TREC  | TOA   | SAMSum | LSHT      | Count | PR-en  | PR-zh  | LCC   | ReprB |       |
| Llama-3.1-8B-Instruct       |                    |        |       |                   |        |       |         |               |        |       |       |                   |       |       |        |           |       |        |        |       |       |       |
| Full                        | 29.39              | 44.61  | 56.34 | 63.63             | 57.48  | 48.08 | 32.28   | 34.43         | 34.51  | 25.29 | 26.80 | 17.37             | 72.50 | 91.48 | 43.59  | 46.50     | 11.10 | 100.00 | 90.45  | 62.93 | 56.19 | 49.76 |
| MInference                  | 28.13              | 42.36  | 53.67 | 62.08             | 56.28  | 46.72 | 32.01   | 33.41         | 33.28  | 25.16 | 27.02 | 16.84             | 69.50 | 90.83 | 42.87  | 43.00     | 6.00  | 96.00  | 84.00  | 60.41 | 55.92 | 47.88 |
| FlexPrefill                 | 27.62              | 42.67  | 52.98 | 62.34             | 57.14  | 45.32 | 30.97   | 33.72         | 33.87  | 24.58 | 26.34 | 17.02             | 67.50 | 90.36 | 44.72  | 43.00     | 6.00  | 94.00  | 83.33  | 61.72 | 55.83 | 47.67 |
| XAttention                  | 28.52              | 43.12  | 54.01 | 60.19             | 56.73  | 45.98 | 32.17   | 33.15         | 34.55  | 25.37 | 27.21 | 17.41             | 71.00 | 91.37 | 44.05  | 44.00     | 6.88  | 95.50  | 85.68  | 61.05 | 55.36 | 48.25 |
| FlashPrefill                | 28.31              | 42.89  | 54.23 | 61.97             | 57.01  | 45.76 | 31.96   | 33.60         | 34.02  | 24.83 | 26.47 | 17.13             | 71.00 | 91.12 | 44.21  | 46.50     | 7.00  | 95.50  | 83.45  | 62.33 | 53.41 | 48.22 |
| FlashPrefill V2             | 30.21              | 44.56  | 55.97 | 62.37             | 58.02  | 48.84 | 32.57   | 33.78         | 34.90  | 25.24 | 27.13 | 17.37             | 72.50 | 91.45 | 44.76  | 45.50     | 7.39  | 97.00  | 89.47  | 62.68 | 53.86 | 49.31 |
| FlashPrefill V2-FP8*        | 29.08              | 45.51  | 54.70 | 62.59             | 55.41  | 46.40 | 32.03   | 32.97         | 34.70  | 24.88 | 26.97 | 17.61             | 70.00 | 91.16 | 44.80  | 45.00     | 6.88  | 96.50  | 85.68  | 63.65 | 57.39 | 48.76 |
| Qwen3-4B-Instruct-2507      |                    |        |       |                   |        |       |         |               |        |       |       |                   |       |       |        |           |       |        |        |       |       |       |
| Full                        | 27.97              | 44.98  | 51.12 | 64.59             | 59.39  | 44.18 | 24.85   | 26.17         | 30.68  | 22.62 | 23.93 | 11.73             | 74.00 | 87.74 | 45.55  | 44.00     | 0.75  | 100.00 | 98.04  | 62.99 | 55.56 | 47.66 |
| MInference                  | 25.89              | 41.69  | 48.26 | 62.16             | 54.17  | 42.76 | 22.16   | 24.89         | 29.12  | 21.98 | 23.15 | 11.28             | 72.00 | 87.62 | 44.32  | 42.50     | 0.00  | 91.00  | 94.67  | 60.92 | 54.48 | 45.48 |
| FlexPrefill                 | 25.26              | 41.48  | 48.02 | 61.98             | 54.96  | 43.21 | 22.71   | 25.43         | 30.94  | 22.15 | 24.47 | 11.47             | 70.50 | 86.18 | 45.87  | 41.00     | 0.50  | 94.00  | 94.20  | 61.67 | 56.19 | 45.82 |
| XAttention                  | 24.31              | 42.67  | 49.01 | 63.02             | 54.14  | 43.14 | 22.56   | 25.12         | 29.63  | 22.41 | 23.68 | 11.56             | 72.50 | 86.47 | 45.13  | 41.50     | 0.00  | 96.00  | 93.89  | 62.24 | 54.92 | 45.90 |
| FlashPrefill                | 25.06              | 42.78  | 49.22 | 62.31             | 55.07  | 42.97 | 23.42   | 25.66         | 29.80  | 22.09 | 23.32 | 11.62             | 72.00 | 87.95 | 45.42  | 42.00     | 0.00  | 92.00  | 94.00  | 62.71 | 53.23 | 45.84 |
| FlashPrefill V2             | 27.56              | 43.73  | 51.07 | 63.72             | 56.39  | 43.11 | 23.18   | 26.36         | 30.48  | 22.67 | 23.88 | 11.76             | 74.00 | 87.74 | 45.78  | 41.25     | 0.50  | 96.00  | 97.17  | 63.50 | 56.38 | 46.96 |
| FlashPrefill V2-FP8         | 26.55              | 42.70  | 50.28 | 61.71             | 55.21  | 41.95 | 24.50   | 26.03         | 30.74  | 22.47 | 24.07 | 11.48             | 73.50 | 88.16 | 45.89  | 41.25     | 0.50  | 93.00  | 94.67  | 60.85 | 52.06 | 46.07 |
| Qwen3-30B-A3B-Instruct-2507 |                    |        |       |                   |        |       |         |               |        |       |       |                   |       |       |        |           |       |        |        |       |       |       |
| Full                        | 31.29              | 44.09  | 54.79 | 66.48             | 63.86  | 56.56 | 32.29   | 24.64         | 31.10  | 21.90 | 23.48 | 11.09             | 76.50 | 91.01 | 45.79  | 51.00     | 16.50 | 100.00 | 100.00 | 72.12 | 69.67 | 51.63 |
| MInference                  | 28.62              | 41.08  | 53.06 | 64.28             | 61.21  | 51.01 | 31.71   | 23.52         | 30.53  | 21.07 | 22.79 | 10.72             | 75.00 | 90.16 | 44.96  | 46.00     | 5.00  | 95.00  | 97.67  | 69.83 | 67.52 | 49.08 |
| FlexPrefill                 | 28.07              | 40.96  | 52.41 | 64.07             | 60.72  | 50.91 | 31.65   | 23.97         | 30.02  | 21.62 | 23.86 | 10.88             | 75.50 | 89.37 | 46.38  | 48.50     | 4.50  | 97.00  | 96.00  | 70.74 | 68.93 | 49.34 |
| XAttention                  | 27.98              | 42.86  | 52.83 | 63.82             | 60.93  | 53.16 | 33.01   | 23.81         | 31.31  | 21.28 | 22.93 | 10.97             | 73.00 | 91.14 | 45.92  | 47.50     | 3.00  | 97.00  | 99.00  | 71.42 | 69.01 | 49.61 |
| FlashPrefill                | 28.35              | 41.11  | 53.34 | 64.81             | 61.27  | 51.09 | 30.08   | 24.11         | 30.55  | 21.74 | 23.21 | 11.02             | 77.00 | 90.68 | 46.12  | 46.50     | 5.00  | 96.50  | 99.00  | 71.86 | 68.38 | 49.61 |
| FlashPrefill V2             | 29.74              | 41.58  | 53.82 | 66.80             | 63.66  | 52.62 | 34.11   | 24.94         | 31.00  | 22.04 | 23.45 | 11.13             | 77.00 | 90.99 | 46.59  | 49.00     | 6.50  | 99.50  | 99.50  | 72.51 | 68.91 | 50.73 |
| FlashPrefill V2-FP8         | 30.03              | 41.37  | 55.28 | 66.31             | 61.40  | 51.83 | 30.63   | 24.58         | 31.19  | 21.62 | 23.56 | 10.96             | 77.50 | 89.61 | 46.08  | 47.50     | 8.50  | 98.00  | 99.00  | 72.16 | 69.05 | 50.29 |

Table 2 | Performance comparison on all 21 tasks of LongBench [2], grouped by task category. Results marked with \* (in gray) are measured with direct online FP8 quantization without corrected weights.

#### 4.1. Setup

**Benchmarks and Models.** RULER [10] stresses long-context retrieval and reasoning at controlled sequence lengths, and LongBench [2] covers a diverse suite of realistic long-context tasks. We evaluate on Llama-3.1-8B-Instruct [9], Qwen3-4B-Instruct-2507 [36], and Qwen3-30B-A3B-Instruct-2507 [36].

**Baselines.** We compare against Full Attention [29], MInference [11], FlexPrefill [14], and XAttention [35]. All methods are evaluated under identical hyperparameter settings on NVIDIA H20 GPUs, the most widely deployed inference GPU in our target deployment environments; all measurements in this section are collected on H20 unless otherwise noted. As dense references, we use FlashAttention-2 [5] and an FA3/4-aligned dense kernel [22, 38] that shares the same warp-specialized pipeline as our sparse operator, so that the measured speedups isolate the benefit of sparsity rather than kernel engineering.

**Implementation Details.** The block-sparse operator is compiled with fixed tile sizes: a query tile of 128 rows (kBlockM) and a key tile of 64 columns (kBlockN). For all accuracy evaluations, every model shares a single configuration without per-model tuning: selection block size  $B = 128$ , 256 sink tokens, a local window of 512 tokens, and  $\alpha = 0.1$  in the max-based thresholding. All operator-level results are measured on a single H20 GPU. End-to-end results (Tabs. 4, 5, and 6) are collected from a single SGLang server instance per configuration with tensor parallelism  $TP=4$  on four H20 GPUs and no data parallelism; the decode attention backend is fixed to FA3/4 across all configurations, and only the prefill attention backend varies. The software stack comprises CUDA 12.9 (driver 535.161.08), PyTorch 2.9.1, CUTLASS 4.3 for the attention kernels, and SGLang 0.5.10 [42] for serving.

#### 4.2. Accuracy Results

**RULER.** Tab. 1 reports RULER scores at controlled sequence lengths (left) together with the end-to-end attention operator speedups (right). Across all three models, FlashPrefill V2 stays

| Model         | 4K    | 8K    | 16K   | 32K   | 64K  | 128K |
|---------------|-------|-------|-------|-------|------|------|
| Llama-3.1-8B  | 76.0% | 54.2% | 33.6% | 18.7% | 9.0% | 4.6% |
| Qwen3-4B      | 72.0% | 48.5% | 29.8% | 17.6% | 9.2% | 4.9% |
| Qwen3-30B-A3B | 70.4% | 46.0% | 29.6% | 16.2% | 9.4% | 4.9% |

Table 3 | Attention density of FlashPrefill V2 measured on needle-in-a-haystack inputs at each sequence length.

| BSZ                                | FA3/4 |      |      |       |       |        | FlashPrefill V2 |                 |                 |                  |                  |                  | FlashPrefill V2-FP8 |                 |                 |                 |                  |                  |
|------------------------------------|-------|------|------|-------|-------|--------|-----------------|-----------------|-----------------|------------------|------------------|------------------|---------------------|-----------------|-----------------|-----------------|------------------|------------------|
|                                    | 4K    | 8K   | 16K  | 32K   | 64K   | 128K   | 4K              | 8K              | 16K             | 32K              | 64K              | 128K             | 4K                  | 8K              | 16K             | 32K             | 64K              | 128K             |
| <i>Llama-3.1-8B-Instruct</i>       |       |      |      |       |       |        |                 |                 |                 |                  |                  |                  |                     |                 |                 |                 |                  |                  |
| 1                                  | 0.15  | 0.29 | 0.63 | 1.49  | 3.95  | 11.81  | 0.15<br>(1.02×) | 0.29<br>(1.02×) | 0.58<br>(1.09×) | 1.17<br>(1.28×)  | 2.46<br>(1.60×)  | 5.49<br>(2.15×)  | 0.10<br>(1.53×)     | 0.18<br>(1.63×) | 0.35<br>(1.81×) | 0.71<br>(2.09×) | 1.48<br>(2.66×)  | 3.23<br>(3.66×)  |
| 4                                  | 0.58  | 1.17 | 2.54 | 5.94  | 12.30 | 32.94  | 0.57<br>(1.01×) | 1.14<br>(1.03×) | 2.32<br>(1.09×) | 4.79<br>(1.24×)  | 7.69<br>(1.60×)  | 15.39<br>(2.14×) | 0.35<br>(1.66×)     | 0.68<br>(1.73×) | 1.36<br>(1.87×) | 2.80<br>(2.12×) | 4.49<br>(2.74×)  | 8.84<br>(3.73×)  |
| 8                                  | 1.07  | 2.21 | 4.87 | 9.83  | 21.23 | 56.67  | 1.06<br>(1.01×) | 2.14<br>(1.03×) | 4.39<br>(1.11×) | 7.77<br>(1.27×)  | 13.18<br>(1.61×) | 26.13<br>(2.17×) | 0.67<br>(1.58×)     | 1.39<br>(1.59×) | 2.81<br>(1.73×) | 4.51<br>(2.18×) | 7.66<br>(2.77×)  | 14.93<br>(3.80×) |
| 16                                 | 2.60  | 3.85 | 8.14 | 16.10 | 37.12 | 102.84 | 2.58<br>(1.01×) | 3.59<br>(1.07×) | 7.31<br>(1.11×) | 12.56<br>(1.28×) | 22.93<br>(1.62×) | 46.96<br>(2.19×) | 1.44<br>(1.80×)     | 2.97<br>(1.30×) | 4.62<br>(1.76×) | 7.28<br>(2.21×) | 13.25<br>(2.80×) | 26.82<br>(3.83×) |
| <i>Qwen3-4B-Instruct-2507</i>      |       |      |      |       |       |        |                 |                 |                 |                  |                  |                  |                     |                 |                 |                 |                  |                  |
| 1                                  | 0.09  | 0.19 | 0.43 | 1.13  | 3.35  | 11.10  | 0.09<br>(1.02×) | 0.18<br>(1.07×) | 0.36<br>(1.19×) | 0.77<br>(1.47×)  | 1.71<br>(1.96×)  | 4.12<br>(2.70×)  | 0.07<br>(1.42×)     | 0.12<br>(1.61×) | 0.23<br>(1.87×) | 0.49<br>(2.33×) | 1.08<br>(3.09×)  | 2.44<br>(4.55×)  |
| 4                                  | 0.35  | 0.74 | 1.69 | 4.35  | 10.61 | 31.47  | 0.35<br>(1.01×) | 0.69<br>(1.06×) | 1.43<br>(1.18×) | 3.06<br>(1.42×)  | 5.38<br>(1.97×)  | 11.55<br>(2.72×) | 0.22<br>(1.59×)     | 0.43<br>(1.72×) | 0.88<br>(1.93×) | 1.86<br>(2.34×) | 3.26<br>(3.25×)  | 6.73<br>(4.67×)  |
| 8                                  | 0.64  | 1.38 | 3.25 | 7.40  | 18.25 | 53.84  | 0.63<br>(1.02×) | 1.29<br>(1.07×) | 2.70<br>(1.20×) | 5.05<br>(1.47×)  | 9.19<br>(1.99×)  | 19.35<br>(2.78×) | 0.43<br>(1.49×)     | 0.87<br>(1.59×) | 1.78<br>(1.82×) | 3.06<br>(2.42×) | 5.51<br>(3.31×)  | 11.28<br>(4.77×) |
| 16                                 | 1.07  | 2.31 | 5.53 | 12.20 | 31.73 | 97.26  | 1.05<br>(1.02×) | 2.15<br>(1.08×) | 4.56<br>(1.21×) | 8.18<br>(1.49×)  | 15.84<br>(2.00×) | 34.48<br>(2.82×) | 0.90<br>(1.18×)     | 1.83<br>(1.26×) | 2.95<br>(1.87×) | 4.92<br>(2.48×) | 9.43<br>(3.37×)  | 20.13<br>(4.83×) |
| <i>Qwen3-30B-A3B-Instruct-2507</i> |       |      |      |       |       |        |                 |                 |                 |                  |                  |                  |                     |                 |                 |                 |                  |                  |
| 1                                  | 0.10  | 0.19 | 0.45 | 1.26  | 4.00  | 13.78  | 0.09<br>(1.02×) | 0.18<br>(1.09×) | 0.36<br>(1.26×) | 0.78<br>(1.61×)  | 1.75<br>(2.29×)  | 4.17<br>(3.30×)  | 0.09<br>(1.06×)     | 0.15<br>(1.24×) | 0.30<br>(1.50×) | 0.61<br>(2.07×) | 1.32<br>(3.03×)  | 3.00<br>(4.59×)  |
| 4                                  | 0.26  | 0.73 | 1.76 | 4.86  | 13.05 | 40.44  | 0.24<br>(1.09×) | 0.67<br>(1.10×) | 1.42<br>(1.24×) | 3.11<br>(1.56×)  | 5.70<br>(2.29×)  | 12.16<br>(3.33×) | 0.22<br>(1.18×)     | 0.57<br>(1.29×) | 1.15<br>(1.53×) | 2.41<br>(2.01×) | 4.21<br>(3.10×)  | 8.59<br>(4.71×)  |
| 8                                  | 0.64  | 1.35 | 3.36 | 8.47  | 22.37 | 68.56  | 0.62<br>(1.03×) | 1.22<br>(1.11×) | 2.65<br>(1.27×) | 5.25<br>(1.61×)  | 9.69<br>(2.31×)  | 20.35<br>(3.37×) | 0.55<br>(1.15×)     | 1.02<br>(1.32×) | 2.11<br>(1.59×) | 4.03<br>(2.10×) | 7.13<br>(3.14×)  | 14.35<br>(4.78×) |
| 16                                 | 1.16  | 3.14 | 5.90 | 13.98 | 38.50 | 123.23 | 1.15<br>(1.01×) | 2.92<br>(1.08×) | 4.60<br>(1.28×) | 8.49<br>(1.65×)  | 16.55<br>(2.33×) | 36.21<br>(3.40×) | 1.04<br>(1.12×)     | 1.73<br>(1.81×) | 3.66<br>(1.61×) | 6.47<br>(2.16×) | 12.16<br>(3.17×) | 25.51<br>(4.83×) |

Table 4 | End-to-end time-to-first-token (TTFT, seconds) of Llama-3.1-8B-Instruct, Qwen3-4B-Instruct-2507, and Qwen3-30B-A3B-Instruct-2507 served by SGLang on the needle-in-a-haystack workload. Each entry is averaged over 30 repeated measurements. Parenthesized numbers are speedups over the FA3/4 backend.

within 1.1 points of Full attention on average (87.79 vs 88.82 on Llama-3.1-8B, 86.23 vs 87.06 on Qwen3-4B, and 91.76 vs 92.05 on Qwen3-30B-A3B), and the FP8 variant costs only an additional 0.4 to 1.2 points. The gap is largest at 128K where the density drops below 5% (Tab. 3 lists the measured densities at each sequence length), yet V2 remains within 1.8 points of Full attention. Meanwhile, the sparse operator including index selection already outperforms FA2 at 4K and reaches 27.2× to 27.5× at 128K in BF16 and 47.3× to 47.6× in FP8.

**LongBench.** Tab. 2 reports the per-task results on LongBench. FlashPrefill V2 achieves the highest average among all sparse operators on every model: 49.31 on Llama-3.1-8B (vs. 48.25 for the strongest baseline, XAttention), 46.96 on Qwen3-4B (vs. 45.90), and 50.73 on Qwen3-30B-A3B (vs. 49.61), staying within 0.9 points of full attention. The advantage is most pronounced on retrieval-sensitive synthetic tasks: on English passage retrieval, V2 retains 96.0 to 99.5 points while the baselines drop to 91.0 to 97.0, and on the counting task V2 remains the closest to full attention on all three models. Summarization tasks are nearly lossless, with V2 occasionally

| Rate<br>(req/s)             | Backend             | TTFT (s)      |               | TPOT (ms)   |             | Throughput   |         |
|-----------------------------|---------------------|---------------|---------------|-------------|-------------|--------------|---------|
|                             |                     | P50           | P99           | P50         | P99         | (tok/s)      | (req/s) |
| Qwen3-4B-Instruct-2507      |                     |               |               |             |             |              |         |
| 1                           | FA3/4               | 76.73         | 154.64        | 419         | 1170        | 24.0         | 0.37    |
|                             | FlashPrefill V2     | 17.23 (4.45x) | 48.73 (3.17x) | 258 (1.62x) | 601 (1.94x) | 45.5 (1.90x) | 0.71    |
|                             | FlashPrefill V2-FP8 | 2.79 (27.47x) | 17.27 (8.95x) | 47 (8.84x)  | 487 (2.40x) | 58.4 (2.44x) | 0.91    |
| 4                           | FA3/4               | 79.05         | 161.54        | 513         | 1102        | 23.8         | 0.37    |
|                             | FlashPrefill V2     | 42.28 (1.87x) | 80.34 (2.01x) | 258 (1.99x) | 509 (2.16x) | 47.3 (1.99x) | 0.74    |
|                             | FlashPrefill V2-FP8 | 24.47 (3.23x) | 46.05 (3.51x) | 165 (3.10x) | 331 (3.33x) | 82.3 (3.46x) | 1.29    |
| 8                           | FA3/4               | 79.55         | 161.01        | 463         | 1110        | 23.9         | 0.37    |
|                             | FlashPrefill V2     | 43.83 (1.81x) | 80.12 (2.01x) | 236 (1.96x) | 523 (2.12x) | 48.3 (2.03x) | 0.76    |
|                             | FlashPrefill V2-FP8 | 25.83 (3.08x) | 46.24 (3.48x) | 152 (3.05x) | 302 (3.68x) | 82.2 (3.45x) | 1.28    |
| 16                          | FA3/4               | 81.91         | 160.99        | 481         | 1110        | 23.8         | 0.37    |
|                             | FlashPrefill V2     | 43.22 (1.90x) | 78.68 (2.05x) | 263 (1.83x) | 501 (2.21x) | 48.9 (2.05x) | 0.76    |
|                             | FlashPrefill V2-FP8 | 25.14 (3.26x) | 45.16 (3.56x) | 152 (3.17x) | 285 (3.89x) | 85.5 (3.59x) | 1.34    |
| Qwen3-30B-A3B-Instruct-2507 |                     |               |               |             |             |              |         |
| 1                           | FA3/4               | 90.69         | 195.43        | 582         | 1335        | 19.9         | 0.31    |
|                             | FlashPrefill V2     | 17.63 (5.14x) | 52.15 (3.75x) | 241 (2.42x) | 604 (2.21x) | 44.5 (2.24x) | 0.70    |
|                             | FlashPrefill V2-FP8 | 7.93 (11.44x) | 21.92 (8.91x) | 102 (5.69x) | 637 (2.10x) | 56.1 (2.82x) | 0.88    |
| 4                           | FA3/4               | 101.44        | 195.33        | 462         | 1312        | 19.9         | 0.31    |
|                             | FlashPrefill V2     | 46.26 (2.19x) | 85.57 (2.28x) | 255 (1.81x) | 520 (2.52x) | 45.5 (2.29x) | 0.71    |
|                             | FlashPrefill V2-FP8 | 30.62 (3.31x) | 57.43 (3.40x) | 198 (2.33x) | 407 (3.22x) | 64.6 (3.25x) | 1.01    |
| 8                           | FA3/4               | 105.52        | 195.34        | 481         | 1333        | 19.9         | 0.31    |
|                             | FlashPrefill V2     | 45.73 (2.31x) | 83.04 (2.35x) | 234 (2.05x) | 529 (2.52x) | 47.0 (2.36x) | 0.73    |
|                             | FlashPrefill V2-FP8 | 33.17 (3.18x) | 58.46 (3.34x) | 165 (2.92x) | 369 (3.62x) | 66.9 (3.36x) | 1.05    |
| 16                          | FA3/4               | 99.68         | 195.20        | 547         | 1347        | 19.9         | 0.31    |
|                             | FlashPrefill V2     | 45.20 (2.21x) | 81.95 (2.38x) | 266 (2.06x) | 514 (2.62x) | 47.4 (2.38x) | 0.74    |
|                             | FlashPrefill V2-FP8 | 32.47 (3.07x) | 57.33 (3.40x) | 185 (2.95x) | 359 (3.75x) | 67.7 (3.41x) | 1.06    |

Table 5 | Open-loop serving results on Qwen3-4B-Instruct-2507 and Qwen3-30B-A3B-Instruct-2507 under Poisson arrivals with mixed prompt lengths (4K–128K RULER needle-in-a-haystack, 100 requests per cell, 64 output tokens each). Parenthesized numbers are speedups over the FA3/4 backend.

exceeding full attention (e.g., 44.76 vs. 43.59 on SAMSum for Llama-3.1-8B). The FP8 variant, even with direct online quantization, still outperforms every baseline on average (48.76/46.07/50.29). Tab. 3 lists the attention densities measured on needle-in-a-haystack inputs.

### 4.3. Efficiency Results

The operator-level speedups over FA2 are reported on the right side of Tab. 1. Fig. 2 further compares against the FA3/4-aligned dense kernel and prior sparse operators at batch size 4: at 128K context, our operator attains 27.2 $\times$  (BF16) and 47.3 $\times$  (FP8) over FA2, 17.5 $\times$  and 30.5 $\times$  over the FA3/4-aligned dense kernel, and clearly surpasses prior sparse operators (18.7 $\times$  for FlashPrefill, 5.4 $\times$  for FlexPrefill, 3.4 $\times$  for XAttention, and 2.8 $\times$  for MInference). The density stays around 70% at 4K and drops to about 5% at 128K (Tab. 3), which is the source of the growing speedup.

**End-to-End Serving Latency.** Tab. 4 reports the end-to-end time-to-first-token of FlashPrefill V2 deployed in SGLang, with batch sizes from 1 to 16 under continuous batching. The operator-

| Chunk size                  | Backend             | TTFT (s)      |                | TPOT (ms)   |              | Throughput   |         |
|-----------------------------|---------------------|---------------|----------------|-------------|--------------|--------------|---------|
|                             |                     | P50           | P99            | P50         | P99          | (tok/s)      | (req/s) |
| Qwen3-4B-Instruct-2507      |                     |               |                |             |              |              |         |
| 8K                          | FA3/4               | 80.98         | 165.14         | 597         | 1228         | 22.5         | 0.35    |
|                             | FlashPrefill V2     | 50.15 (1.61x) | 95.23 (1.73x)  | 360 (1.66x) | 684 (1.80x)  | 38.7 (1.72x) | 0.60    |
|                             | FlashPrefill V2-FP8 | 33.47 (2.42x) | 61.20 (2.70x)  | 259 (2.30x) | 529 (2.32x)  | 60.7 (2.70x) | 0.95    |
| 16K                         | FA3/4               | 79.42         | 161.96         | 580         | 1203         | 22.9         | 0.36    |
|                             | FlashPrefill V2     | 45.09 (1.76x) | 85.47 (1.89x)  | 315 (1.84x) | 615 (1.96x)  | 43.9 (1.91x) | 0.69    |
|                             | FlashPrefill V2-FP8 | 26.80 (2.96x) | 50.34 (3.22x)  | 193 (3.00x) | 382 (3.15x)  | 73.3 (3.20x) | 1.15    |
| Qwen3-30B-A3B-Instruct-2507 |                     |               |                |             |              |              |         |
| 8K                          | FA3/4               | 96.61         | 199.39         | 693         | 1494         | 18.7         | 0.29    |
|                             | FlashPrefill V2     | 79.81 (1.21x) | 149.52 (1.33x) | 574 (1.21x) | 1200 (1.24x) | 24.6 (1.32x) | 0.38    |
|                             | FlashPrefill V2-FP8 | 63.23 (1.53x) | 121.53 (1.64x) | 505 (1.37x) | 1009 (1.48x) | 29.2 (1.56x) | 0.46    |
| 16K                         | FA3/4               | 94.78         | 195.84         | 672         | 1466         | 19.0         | 0.30    |
|                             | FlashPrefill V2     | 46.47 (2.04x) | 88.15 (2.22x)  | 323 (2.08x) | 652 (2.25x)  | 42.1 (2.21x) | 0.66    |
|                             | FlashPrefill V2-FP8 | 33.72 (2.81x) | 62.19 (3.15x)  | 250 (2.69x) | 464 (3.16x)  | 57.1 (3.00x) | 0.89    |

Table 6 | Open-loop serving at 16 req/s with chunked prefill at chunk sizes of 8K and 16K (same workload as Tab. 5, where chunking is disabled). Parenthesized numbers are speedups over the FA3/4 backend at the same chunk size.

level gains translate directly into serving latency: at 128K context, FlashPrefill V2 reduces TTFT by 2.1× to 3.4× in BF16 and 3.7× to 4.8× in FP8 across all three models and batch sizes (e.g., from 123.2s to 36.2s in BF16 and 25.5s in FP8 at batch size 16 on Qwen3-30B-A3B). The speedup is nearly batch-invariant, indicating that the sparse operator scales with continuous batching in the same way as the dense backend. As expected, the end-to-end speedup is bounded by the attention share of total prefill compute and is therefore smaller than the operator-level one; it grows with sequence length as attention becomes dominant, and is largest on Qwen3-30B-A3B, where attention accounts for a larger fraction of prefill compute. At 4K, where the density remains around 70%, BF16 stays at parity with FA3/4 (1.01× to 1.09×), where the modest attention savings are largely offset by the index stage overhead, while FP8 already delivers 1.1× to 1.8×.

**Open-Loop Serving.** Tab. 5 moves from fixed batches to an open-loop setting: requests arrive as a Poisson process at 1 to 16 req/s with prompt lengths mixed over 4K–128K, so prefills and decodes of different requests continuously share the engine. The FA3/4 backend is saturated at every rate (P50 TTFT of 77–106 s, request throughput capped at 0.31–0.37 req/s), whereas FlashPrefill V2 sustains about twice the request throughput (0.70–0.76 req/s) and cuts the P50 TTFT to 17–46 s; with FP8 the request throughput rises to 0.88–1.34 req/s and the P50 TTFT drops further to 2.8–33 s. The speedup is largest at the lowest rate (4.5×–5.1× P50 at 1 req/s in BF16 and 11.4×–27.5× in FP8), where faster prefills shorten queueing much more than at higher rates, and remains 1.8×–2.3× in BF16 and 3.1×–3.3× in FP8 once the system is loaded. Shorter prefills also benefit decoding: because long dense prefills block the decode steps of concurrently running requests, P50 TPOT improves by 1.6×–2.4× in BF16 and 2.3×–8.8× in FP8, and output throughput rises by 1.9×–2.4× and 2.4×–3.6× respectively.

**Compatibility with Chunked Prefill.** Chunked prefill [1], which bounds the number of tokens processed per scheduling step, is the standard mechanism in production engines for protecting inter-token latency from long prefills. Tab. 6 repeats the open-loop experiment at 16 req/s with chunk sizes of 8K and 16K. Chunking barely affects the dense backend’s TTFT, but it erodes our

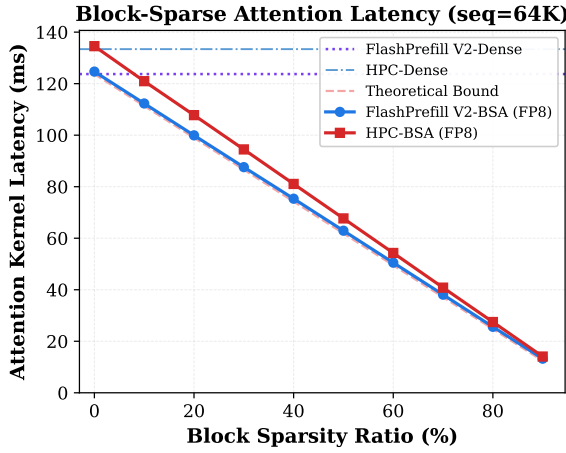


Figure 5 | Block-sparse attention latency at 64K sequence length in FP8. HPC-BSA does not support BF16, so only FP8 is compared. Both sparse operators track the theoretical bound; FlashPrefill V2 is 6 to 7% faster than HPC-BSA at every sparsity level.

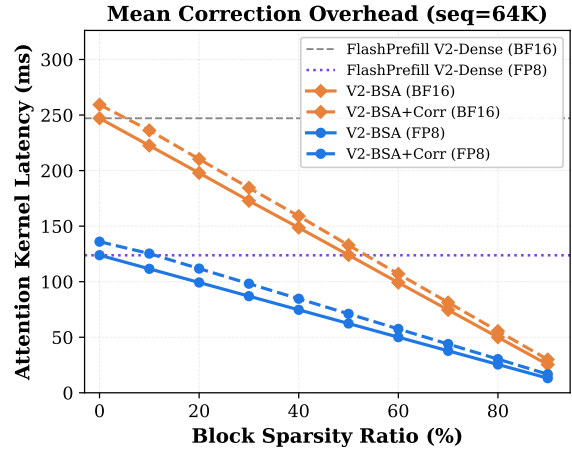


Figure 6 | Overhead of mean correction at 64K sequence length. The corrected variants (dashed) stay close to the uncorrected ones (solid) in both BF16 and FP8, and the absolute overhead shrinks as sparsity increases.

speedup because the index selection is re-run on every chunk and the mandatory tail blocks raise the effective density of short chunks: the P50 TTFT speedup in BF16 drops from  $1.9\times$  without chunking to  $1.6\times$  at 8K on Qwen3-4B, and from  $2.2\times$  to  $1.2\times$  on Qwen3-30B-A3B. Even so, FlashPrefill V2 remains faster than the dense backend at 8K ( $1.2\times$ – $1.6\times$  in BF16 and  $1.5\times$ – $2.4\times$  in FP8), and a 16K chunk recovers most of the margin ( $1.8\times$ – $2.0\times$  and  $2.8\times$ – $3.0\times$  respectively), so we recommend chunk sizes of at least 8K when deploying FlashPrefill V2 together with chunked prefill.

**Comparison with the Production Kernel.** Fig. 5 compares our block-sparse operator against the block-sparse attention (BSA) kernel in HPC-Ops [26], a production inference operator library. Since the HPC-Ops BSA kernel only supports FP8, the comparison is conducted in FP8 at 64K sequence length. Our operator is consistently faster than HPC-BSA across all sparsity levels (by 6 to 7%), and both dense references are comparable (123.7 ms vs 133.4 ms), confirming that the gain comes from the sparse execution path rather than the dense backbone. Tab. 7 summarizes the design differences behind this gap.

**Overhead of Mean Correction.** Fig. 6 quantifies the runtime cost of the correction term at 64K. Because all pruned blocks within a query tile are replaced by a single pass over their pooled K/V means, the extra latency is small and nearly constant in absolute terms: at most 12.2 ms in the dense limit and only 3.6 to 4.5 ms at 90% sparsity. The relative overhead therefore stays within 5% to 14% up to 50% sparsity, and reaches 18% (BF16) to 27% (FP8) only at 90% sparsity, the regime where the correction is most needed for accuracy.

#### 4.4. Ablation Study

**Effect of Mean Correction.** Tab. 8 ablates the mean correction term (Sec. 3.2) on RULER with Qwen3-4B-Instruct-2507, comparing the full FlashPrefill V2 pipeline against a variant in which all unselected blocks are simply discarded. Without correction, the dropped probability mass grows as the density falls, so the accuracy gap widens with sequence length: in BF16 the variant

|                           | HPC-Ops BSA                    | FlashPrefill V2                     |
|---------------------------|--------------------------------|-------------------------------------|
| Sparse index format       | dense mask, padded, per Q head | CSR of selected blocks, per KV head |
| Index memory @ 64K / 128K | 8 MB / 32 MB (any density)     | 3 MB / 6 MB (9% / 5% density)       |
| GEMM-softmax overlap      | serial                         | intra-warpgroup pingpong            |
| Paged KV addressing       | per-page TMA (page size   128) | cp.async, arbitrary page size       |
| KV splitting              | none                           | count-balanced CSR splits           |
| Mean correction           | none                           | fused into the mainloop             |
| Precision                 | FP8 only                       | BF16 + FP8                          |

Table 7 | Design comparison with the HPC-Ops block-sparse attention kernel. Index memory is evaluated at batch size 1 with 32 query heads and 4 KV heads: the dense mask stores one byte per (query head, 128-token Q tile, 128-token KV tile) regardless of density, whereas the CSR index stores only the selected 64-token tiles as int32 at the measured needle-in-a-haystack densities (Tab. 3), is organized per KV head under PackGQA, and matches the SGLang index format. The fused correction path has no counterpart in HPC-Ops BSA.

| Method                | 4K    | 8K    | 16K   | 32K   | 64K   | 128K  | Avg.  |
|-----------------------|-------|-------|-------|-------|-------|-------|-------|
| Full Attention        | 94.82 | 92.98 | 91.68 | 88.86 | 82.81 | 71.22 | 87.06 |
| V2 w/o correction     | 94.38 | 92.06 | 91.18 | 87.21 | 80.92 | 68.86 | 85.77 |
| FlashPrefill V2       | 94.49 | 92.42 | 91.56 | 87.48 | 81.67 | 69.76 | 86.23 |
| V2-FP8 w/o correction | 93.78 | 91.82 | 90.68 | 85.04 | 76.86 | 62.78 | 83.49 |
| FlashPrefill V2-FP8   | 94.13 | 92.37 | 91.36 | 87.47 | 80.59 | 68.97 | 85.82 |

Table 8 | Ablation of the mean correction term on RULER with Qwen3-4B-Instruct-2507, in both BF16 and FP8. Discarding unselected blocks without compensation degrades accuracy increasingly toward longer contexts, while the zero-order correction stays close to full attention at all lengths in both precisions.

trails the corrected pipeline by only 0.3 points on average up to 32K, but by 0.8 points at 64K and 0.9 points at 128K, where fewer than 5% of the blocks survive selection (Tab. 3). Restoring the correction term recovers nearly all of this degradation at a negligible runtime cost (Fig. 6), keeping FlashPrefill V2 within 0.9 points of full attention on average and within 1.5 points even at 128K. The correction becomes substantially more important in FP8: without it, the FP8 variant loses 2.3 points on average and 6.2 points at 128K relative to its corrected counterpart, compared with only 0.5 and 0.9 points in BF16. We attribute this to the compressed score margins under quantization, which let the blocks discarded by the max-based threshold carry a larger share of the softmax mass, so recovering them matters more. With correction enabled, the FP8 pipeline tracks BF16 closely (85.82 vs. 86.23 on average), confirming that the correction remains effective even when the pooled statistics are quantized.

**Effect of the Selection Threshold.** Tab. 9 sweeps the threshold  $\alpha$  of the max-based selection on RULER at 64K with Qwen3-4B-Instruct-2507 in FP8, and reports the measured attention density together with the scores with and without mean correction. As  $\alpha$  decreases from 0.2 to 0.0125, the density rises from 5.2% to 23.6%. The corrected FP8 pipeline is remarkably flat across the entire sweep, staying within 2.7 points of full attention (82.81) even at 5.2% density and within 2.3 points at the default  $\alpha = 0.1$ , whereas removing the correction consistently costs 2.7 to 5.4 further points, with the gap widening as the density falls. Mean correction therefore enlarges the usable sparsity range: with compensation,  $\alpha$  can be pushed to 0.2 at 64K—halving the density relative to the default—while retaining 80.12 points.

| $\alpha$              | 0.2                   | 0.1                   | 0.05                  | 0.025                 | 0.0125                |
|-----------------------|-----------------------|-----------------------|-----------------------|-----------------------|-----------------------|
| Density (%)           | 5.2                   | 9.2                   | 14.1                  | 19.8                  | 23.6                  |
| V2-FP8 w/o correction | 74.68( <b>-5.44</b> ) | 76.86( <b>-3.73</b> ) | 77.16( <b>-3.51</b> ) | 77.41( <b>-3.31</b> ) | 78.02( <b>-2.73</b> ) |
| FlashPrefill V2-FP8   | 80.12                 | 80.59                 | 80.67                 | 80.72                 | 80.75                 |

Table 9 | Ablation of the selection threshold  $\alpha$  on RULER at 64K with Qwen3-4B-Instruct-2507 in FP8. Density is the fraction of selected attention blocks. **Red numbers** are the score drops of the uncorrected variant relative to the corrected pipeline at the same  $\alpha$ . Full attention scores 82.81.

## 5. Conclusion

In this paper, we present FlashPrefill V2, which evolves FlashPrefill from an algorithmic prototype toward practical long-context prefilling. A zero-order mean correction term compensates pruned blocks with their pooled K/V statistics inside the softmax computation, confining the accuracy loss to within about one point on RULER and LongBench averages, and to within 1.8 points at 128K on RULER where fewer than 5% of the blocks are computed, while proving particularly effective under FP8 quantization. The redesigned sparse operator aligns with the FlashAttention-3/4 execution model—PackGQA memory access, warp-specialized producer-consumer pipelines, intra-warpgroup pingpong overlap, and FP8 execution—attaining up to **27.19×** (BF16) and **47.26×** (FP8) speedups over FlashAttention-2 at 128K, and a **30.49×** speedup over an FA3/4-aligned dense baseline. With native support for paged KV cache and continuous batching, FlashPrefill V2 integrates into SGLang as a standard attention backend without any model-side changes, reducing end-to-end time-to-first-token by up to 3.4× (BF16) and 4.8× (FP8) at 128K under continuous batching. We hope this work helps bring sparse attention from algorithmic exploration toward practical serving.

## References

- [1] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. Gulavani, A. Tumanov, and R. Ramjee. Taming throughput-latency tradeoff in llm inference with sarathi-serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.
- [2] Y. Bai, X. Lv, J. Zhang, H. Lyu, J. Tang, Z. Huang, Z. Du, X. Liu, A. Zeng, L. Hou, Y. Dong, J. Tang, and J. Li. Longbench: A bilingual, multitask benchmark for long context understanding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- [3] I. Beltagy, M. E. Peters, and A. Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020. URL <https://arxiv.org/abs/2004.05150>.
- [4] Z. Cai, Y. Zhang, B. Gao, Y. Liu, Y. Li, T. Liu, K. Lu, W. Xiong, Y. Dong, J. Hu, and W. Xiao. Pyramidkv: Dynamic kv cache compression based on pyramidal information funneling. *arXiv preprint arXiv:2406.02069*, 2024.
- [5] T. Dao. Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*, 2023. URL <https://arxiv.org/abs/2307.08691>.
- [6] Q. Fan, H. Huang, Y. Ai, and R. He. Rectifying magnitude neglect in linear attention. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025.

- [7] Q. Fan, H. Huang, Z. Wu, B. Wang, and R. He. Uniprefill: Universal long-context prefill acceleration via block-wise dynamic sparsification. *arXiv preprint arXiv:2605.06221*, 2026.
- [8] Q. Fan, H. Huang, Z. Wu, J. Wang, B. Wang, and R. He. Flashprefill: Instantaneous pattern discovery and thresholding for ultra-fast long-context prefilling. *arXiv preprint arXiv:2603.06199*, 2026.
- [9] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- [10] C.-P. Hsieh, S. Sun, S. Krizan, S. Acharya, D. Rekesh, F. Jia, Y. Zhang, and B. Ginsburg. Ruler: What’s the real context size of your long-context language models? *arXiv preprint arXiv:2404.06654*, 2024. URL <https://arxiv.org/abs/2404.06654>.
- [11] H. Jiang, Y. Li, C. Zhang, Q. Wu, X. Luo, S. Ahn, Z. Han, A. H. Abdi, D. Li, C.-Y. Lin, Y. Yang, and L. Qiu. MInference 1.0: Accelerating pre-filling for long-context LLMs via dynamic sparse attention. In *Advances in Neural Information Processing Systems*, 2024.
- [12] N. Kitaev, Ł. Kaiser, and A. Levskaya. Reformer: The efficient transformer. *arXiv preprint arXiv:2001.04451*, 2020.
- [13] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.
- [14] X. Lai, J. Lu, Y. Luo, Y. Ma, and X. Zhou. Flexprefill: A context-aware sparse attention mechanism for efficient long-sequence inference. In *International Conference on Learning Representations*, 2025.
- [15] H. Li, Y. Li, J. Chen, T. Ye, H. Liu, J. Yu, D. Wang, R. Zhang, Z. Xie, E. Xie, and S. Han. Sol-attn: Accelerating video generation inference via on-the-fly attention sparsification. *arXiv preprint arXiv:2607.24027*, 2026.
- [16] Y. Li, Y. Huang, B. Yang, B. Venkitesh, A. Locatelli, H. Ye, T. Cai, P. Lewis, and D. Chen. Snapkv: Llm knows what you are looking for before generation. *arXiv preprint arXiv:2404.14469*, 2024.
- [17] Y. Li, H. Jiang, C. Zhang, Q. Wu, X. Luo, S. Ahn, A. H. Abdi, D. Li, J. Gao, Y. Yang, and L. Qiu. Mminference: Accelerating pre-filling for long-context vlms via modality-aware permutation sparse attention. *arXiv preprint arXiv:2504.16083*, 2025.
- [18] E. Lu, Z. Jiang, J. Liu, Y. Du, T. Jiang, C. Hong, S. Liu, W. He, E. Yuan, Y. Wang, Z. Huang, H. Yuan, S. Xu, X. Xu, G. Lai, Y. Chen, H. Zheng, J. Yan, J. Su, Y. Wu, N. Y. Zhang, Z. Yang, X. Zhou, M. Zhang, and J. Qiu. Moba: Mixture of block attention for long-context llms. *arXiv preprint arXiv:2502.13189*, 2025. URL <https://arxiv.org/abs/2502.13189>.
- [19] B. Peng, R. Zhang, D. Goldstein, E. Alcaide, X. Du, H. Hou, J. Lin, J. Liu, J. Lu, W. Merrill, G. Song, K. Tan, S. Utpala, N. Wilce, J. S. Wind, T. Wu, D. Wuttke, and C. Zhou-Zheng. Rwkv-7 "goose" with expressive dynamic state evolution. *arXiv preprint arXiv:2503.14456*, 2025.
- [20] R. Qin, Z. Li, W. He, M. Zhang, Y. Wu, W. Zheng, and X. Xu. Mooncake: A kvcache-centric disaggregated architecture for llm serving. *arXiv preprint arXiv:2407.00079*, 2024.

- [21] L. Ribar, I. Chelombiev, L. Hudlass-Galley, C. Blake, C. Luschi, and D. Orr. Sparq attention: Bandwidth-efficient llm inference. *arXiv preprint arXiv:2312.04985*, 2023.
- [22] J. Shah, G. Bikshandi, Y. Zhang, V. Thakkar, P. Ramade, and T. Dao. Flashattention-3: Fast and accurate attention with asynchrony and low-precision. *arXiv preprint arXiv:2407.08608*, 2024.
- [23] H. Sun, Z. Chen, X. Yang, Y. Tian, and B. Chen. Triforce: Lossless acceleration of long sequence generation with hierarchical speculative decoding. *arXiv preprint arXiv:2404.11912*, 2024.
- [24] Y. Sun, L. Dong, S. Huang, S. Ma, Y. Xia, J. Xue, J. Wang, and F. Wei. Retentive network: A successor to transformer for large language models. *arXiv preprint arXiv:2307.08621*, 2023.
- [25] J. Tang, Y. Zhao, K. Zhu, G. Xiao, B. Kasikci, and S. Han. Quest: Query-aware sparsity for efficient long-context llm inference. *arXiv preprint arXiv:2406.10774*, 2024.
- [26] Tencent Hunyuan AI Infra Team. HPC-Ops: High performance computing operators for large language model inference. <https://github.com/Tencent/hpc-ops>, 2026. Accessed: 2026-08-17.
- [27] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- [28] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.
- [29] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017.
- [30] A. Vyas, A. Katharopoulos, and F. Fleuret. Fast transformers with clustered attention. *arXiv preprint arXiv:2007.04825*, 2020.
- [31] C. Xiao, P. Zhang, X. Han, G. Xiao, Y. Lin, Z. Zhang, Z. Liu, and M. Sun. Inllm: Training-free long-context extrapolation for llms with an efficient context memory. *arXiv preprint arXiv:2402.04617*, 2024.
- [32] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis. Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453*, 2023.
- [33] G. Xiao, J. Tang, J. Zuo, J. Guo, S. Yang, H. Tang, Y. Fu, and S. Han. Duoattention: Efficient long-context llm inference with retrieval and streaming heads. *arXiv preprint arXiv:2410.10819*, 2024.
- [34] G. Xiao, J. Guo, K. Mazaheri, and S. Han. Optimizing mixture of block attention. *arXiv preprint arXiv:2511.11571*, 2025. URL <https://arxiv.org/abs/2511.11571>.
- [35] R. Xu, G. Xiao, H. Huang, J. Guo, and S. Han. Xattention: Block sparse attention with antidiagonal scoring. In *International Conference on Machine Learning*, 2025.

- [36] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, C. Zheng, D. Liu, F. Zhou, F. Huang, F. Hu, H. Ge, H. Wei, H. Lin, J. Tang, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Zhou, J. Lin, K. Dang, K. Bao, K. Yang, L. Yu, L. Deng, M. Li, M. Xue, M. Li, P. Zhang, P. Wang, Q. Zhu, R. Men, R. Gao, S. Liu, S. Luo, T. Li, T. Tang, W. Yin, X. Ren, X. Wang, X. Zhang, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Zhang, Y. Wan, Y. Liu, Z. Wang, Z. Cui, Z. Zhang, Z. Zhou, and Z. Qiu. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. URL <https://arxiv.org/abs/2505.09388>.
- [37] J. Yuan, H. Gao, D. Dai, J. Luo, L. Zhao, Z. Zhang, Z. Xie, Y. Wei, L. Wang, Z. Xiao, Y. Wang, C. Ruan, M. Zhang, W. Liang, and W. Zeng. Native sparse attention: Hardware-aligned and natively trainable sparse attention. In *Annual Meeting of the Association for Computational Linguistics*, 2025.
- [38] T. Zadori, M. Hoehnerbach, J. Shah, T. Liu, V. Thakkar, and T. Dao. Flashattention-4: Algorithm and kernel pipelining co-design for asymmetric hardware scaling. *arXiv preprint arXiv:2603.05451*, 2026.
- [39] M. Zaheer, G. Guruganesh, A. Dubey, J. Ainslie, C. Alberti, S. Ontanon, P. Pham, A. Ravula, Q. Wang, L. Yang, and A. Ahmed. Big bird: Transformers for longer sequences. *arXiv preprint arXiv:2007.14062*, 2020.
- [40] Z. Zhang, Y. Sheng, et al. H2o: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*, 2023.
- [41] W. Zhao, Z. Zhou, Z. Su, C. Xiao, Y. Li, Y. Li, Y. Zhang, W. Zhao, Z. Li, Y. Huang, A. Sun, X. Han, and Z. Liu. Infilmm-v2: Dense-sparse switchable attention for seamless short-to-long adaptation. *arXiv preprint arXiv:2509.24663*, 2025. URL <https://arxiv.org/abs/2509.24663>.
- [42] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng. Sglang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems*, 2024.
- [43] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang. Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving. *arXiv preprint arXiv:2401.09670*, 2024.
- [44] K. Zhu, T. Tang, Q. Xu, Y. Gu, Z. Zeng, R. Kadekodi, L. Zhao, A. Li, A. Krishnamurthy, and B. Kasikci. Tactic: Adaptive sparse attention with clustering and distribution fitting for long-context llms. *arXiv preprint arXiv:2502.12216*, 2025.